Zain Jordan SQL Agent

Hugging Face Spaces Deployment Guide

Student handout for deploying the Gradio SQL Agent app with SQLite, LangChain SQLDatabaseToolkit, OpenAI gpt-4.1-mini, and Hugging Face Spaces.

1. What We Are Deploying

We are deploying a simple Gradio SQL Agent application on Hugging Face Spaces. The application allows a user to ask business questions from the Zain Jordan Customer 360 SQLite database.

- Gradio provides the user interface.
- SQLite stores the Customer 360 database.
- LangChain SQLDatabase connects the app to the SQLite database.
- LangChain SQLDatabaseToolkit gives the agent database tools.
- OpenAI gpt-4.1-mini is used as the default model.
- Hugging Face Spaces hosts the application.

> Important: this first deployment is intentionally minimal. There is no database upload button, no API key textbox, and no model selector in the UI. The database is placed inside the Space files, and the API key is stored securely as a Space Secret.

2. Final Folder Structure

The final Hugging Face Space should have this structure:

```
zain-sql-agent-space/
|
■■■ app.py
■■■ requirements.txt
■■■ README.md
|
■■■ database_/
■■■ zain_customer_360_ai_demo.db
```

The most important path is:

```
database_/zain_customer_360_ai_demo.db
```

3. Step-by-Step Deployment Process

Step 1 - Create a Hugging Face Space

- Go to Hugging Face and create a new Space.
- Choose Gradio as the SDK.
- Use CPU Basic hardware. This is enough because the LLM runs through the OpenAI API.
- Use Private visibility if the database should not be publicly visible.

Step 2 - Create the Required Files

- Create app.py.
- Create requirements.txt.
- Create README.md.
- Create a folder named database_.
- Upload zain_customer_360_ai_demo.db inside database_.

Step 3 - Add OpenAI API Key as a Secret

- Open the Space Settings.

- Go to Variables and secrets.
- Create a new Secret named OPENAI_API_KEY.
- Paste your OpenAI API key as the value.
- Save and rebuild or restart the Space.

Step 4 - Add requirements.txt

- Paste the requirements.txt content from this guide.
- Hugging Face will install these packages when the Space builds.

Step 5 - Add app.py

- Paste the full app.py code from this guide.
- Make sure DB_PATH points to database_/zain_customer_360_ai_demo.db.
- Do not hard-code the OpenAI API key in app.py.

Step 6 - Build and Test

- Wait for the Space to finish building.
- Open the App tab.
- Ask one of the test questions from this guide.
- Check the output and Space logs if there is an error.

4. requirements.txt

Create a file named requirements.txt and paste this content:

```
gradio
pandas
sqlalchemy
langchain
langchain-openai
langchain-community
openai
```

5. Full app.py Code

Create a file named app.py and paste this full code:

```
import os
import traceback
from pathlib import Path

import gradio as gr

from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import SQLDatabaseToolkit


# ============================================================
# Fixed configuration
# ============================================================

DB_PATH = Path("database_") / "zain_customer_360_ai_demo.db"
MODEL_NAME = "gpt-4.1-mini"


# ============================================================
# SQL Agent system prompt
# ============================================================

SYSTEM_PROMPT = """
You are a telecom business intelligence SQL agent for Zain Jordan.

You are connected to a SQLite Customer 360 database.

Rules:
- Use SELECT queries only.
- Never modify the database.
- Never use INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, REPLACE, ATTACH, DETACH,
or VACUUM.
- Inspect the database schema before writing SQL.
- Limit results to 10 rows unless the user asks otherwise.
- Use only relevant columns.
- Do not guess facts that are not in the database.
- Explain the result in simple business language.

Useful context:
- Churn analysis usually uses customers and customer_churn_scores.
- Revenue/value analysis usually uses customer_value_segments, invoices, payments, or
transactions.
- Complaint analysis usually uses complaints and support_interactions.
- Campaign analysis usually uses campaigns and customer_campaign_responses.
- Network analysis usually uses network_towers and network_events.

Final answer format:
1. Direct Answer
2. Key Numbers
3. Business Interpretation
4. Recommended Next Action
""".strip()


# ============================================================
# Cache the agent so it is not rebuilt on every click
# ============================================================

sql_agent = None
```

```
def get_sql_agent():
"""
Build the SQL Agent once and reuse it.
"""

global sql_agent

if sql_agent is not None:
return sql_agent

api_key = os.environ.get("OPENAI_API_KEY")

if not api_key:
raise ValueError(
"OPENAI_API_KEY is missing. Add it in Hugging Face Space Settings -> Variables
and secrets -> Secrets."
)

if not DB_PATH.exists():
raise FileNotFoundError(
f"Database not found at: {DB_PATH}\n\n"
"Please create a folder named database_ and upload zain_customer_360_ai_demo.db
inside it."
)

db_uri = f"sqlite:///{DB_PATH.resolve()}"

db = SQLDatabase.from_uri(db_uri)

llm = init_chat_model(
MODEL_NAME,
model_provider="openai"
)

toolkit = SQLDatabaseToolkit(
db=db,
llm=llm
)

tools = toolkit.get_tools()

sql_agent = create_agent(
model=llm,
tools=tools,
system_prompt=SYSTEM_PROMPT
)

return sql_agent


def run_sql_agent(question):
"""
Main Gradio function.

Flow:
User question -> SQL Agent -> SQLDatabaseToolkit -> SQLite DB -> Business answer
"""

try:
if not question or not question.strip():
return "Please enter a business question."

agent = get_sql_agent()

result = agent.invoke({
"messages": [
{
"role": "user",
"content": question
}
]
})

final_message = result["messages"][-1]

if hasattr(final_message, "content"):
return final_message.content

return str(final_message)
```

````
except Exception as e:
return f"""
### Error

{str(e)}

### Traceback

```text
{traceback.format_exc()}
```
"""


# ============================================================
# Gradio UI
# ============================================================

with gr.Blocks(title="Zain Jordan SQL Agent") as demo:

gr.Markdown("# Zain Jordan Customer 360 SQL Agent")

gr.Markdown("""
Ask business questions from the Zain Jordan Customer 360 SQLite database.

This app uses:

- SQLite database
- LangChain `SQLDatabase`
- LangChain `SQLDatabaseToolkit`
- OpenAI `gpt-4.1-mini`
- Gradio interface

Expected database location:

```text
database_/zain_customer_360_ai_demo.db
```

Example questions:

- Which cities have the most high-risk churn customers?
- Which customer value segments have the highest average ARPU?
- What are the top complaint categories?
- Which campaigns had the highest conversion rate?
""")

question = gr.Textbox(
label="Ask a business question",
lines=4,
value="Which cities have the most high-risk churn customers? Show the top 10
cities.",
placeholder="Ask a question about churn, revenue, complaints, campaigns, customers,
or network events."
)

run_button = gr.Button("Run SQL Agent", variant="primary")

answer = gr.Markdown(label="Answer")

run_button.click(
fn=run_sql_agent,
inputs=question,
outputs=answer
)


if __name__ == "__main__":
demo.launch()
````

6. Optional README.md

This README is optional, but it helps students understand the deployed Space.

````
# Zain Jordan Customer 360 SQL Agent

This Hugging Face Space runs a simple Gradio SQL Agent app.

## What this app does

The app allows users to ask business questions from the Zain Jordan Customer 360 SQLite
database.

It uses:

- Gradio
- SQLite
- LangChain SQLDatabase
- LangChain SQLDatabaseToolkit
- OpenAI gpt-4.1-mini

## File structure

```text
app.py
requirements.txt
database_/
zain_customer_360_ai_demo.db
```

## Required Secret

Add the following secret in Hugging Face Space Settings:

```text
OPENAI_API_KEY
```

## Example Questions

```text
Which cities have the most high-risk churn customers?
```

```text
Which customer value segments have the highest average ARPU?
```

```text
What are the top complaint categories?
```

```text
Which campaigns had the highest conversion rate?
```
````

7. What Each File Does

File / Folder	Purpose
app.py	Main Gradio app and SQL Agent logic.
requirements.txt	Python packages needed by the Space.
README.md	Explanation for students or users.
database_	Folder where the SQLite database is stored.
zain_customer_360_ai_demo.db	Zain Jordan Customer 360 SQLite database.
OPENAI_API_KEY Secret	Securely stores the OpenAI API key.

8. How the Application Works

The application follows this flow:

```
User Question
↓
Gradio Interface
↓
run_sql_agent()
↓
LangChain SQL Agent
↓
SQLDatabaseToolkit
↓
SQLite Database
↓
Business Answer
```

In simple terms, the user types a question, Gradio sends it to run_sql_agent(), the SQL Agent uses SQLDatabaseToolkit tools to inspect and query the SQLite database, and the LLM explains the result in business language.

9. Testing the App

After the Space builds successfully, open the App tab and test these questions:

Test 1

```
How many customers are in the database?
```

Test 2

```
Which cities have the most high-risk churn customers? Show the top 10 cities.
```

Test 3

```
Which customer value segments have the highest average ARPU and six-month revenue?
```

Test 4

```
What are the top complaint categories?
```

Test 5

```
Which campaigns had the highest conversion rate?
```

10. Common Errors and Fixes

Error	Meaning	Fix
OPENAI_API_KEY is missing	The Space cannot find the OpenAI key.	Add OPENAI_API_KEY in Settings -> Variables and secrets -> Secrets.
Database not found	The database file is not in the expected location.	Make sure the file path is database_/zain_customer_360_ai_demo.db.
App builds but answer fails	The app started, but the agent failed during execution.	Check OpenAI key validity, billing, model access, database file, and Space logs.
Wrong or incomplete answer	The question may be vague or the agent selected an imperfect join.	Ask a clearer question with specific columns, filters, or grouping.

11. Important Teaching Notes

Keep the first version simple. Do not add a database upload button, API key textbox, model dropdown, advanced tracing, authentication, or streaming output in the first deployment. The goal is to show the clean deployment path.

```
app.py + requirements.txt + database_ folder + OPENAI_API_KEY secret
```

Do not hard-code the API key.

Wrong:

```
OPENAI_API_KEY = "sk-..."
```

Correct:

```
os.environ.get("OPENAI_API_KEY")
```

Why we use database_: the app expects this exact path:

```
DB_PATH = Path("database_") / "zain_customer_360_ai_demo.db"
```

12. Final Student Checklist

Checklist Item	Done
Space created with Gradio SDK	■
app.py uploaded	■
requirements.txt uploaded	■
database_ folder created	■
zain_customer_360_ai_demo.db uploaded inside database_	■
OPENAI_API_KEY added as Secret	■
Space rebuilt successfully	■
Test question works	■

13. Final Package

The ZIP package contains:

```
app.py
requirements.txt
```

```
README.md
database_/
UPLOAD_DATABASE_HERE.txt
```

Replace the placeholder file with:

```
database_/zain_customer_360_ai_demo.db
```